

Chapter 18. What's Next for Recs?

We find ourselves in a transitional time for recommendation systems. However, this is quite normal for this field, as it is in many segments of the tech industry. One of the realities of a field that is so closely aligned with business objectives and with such strong capabilities for business value is that the field tends to be constantly searching for any and all opportunities to advance.

In this chapter, we'll briefly introduce some of the modern views of where recommendation systems are going. An important point to consider is that recommendation systems as a science spread both depth first and breadth first simultaneously. Looking at the most cutting-edge research in the field means that you're seeing deep optimization in areas that have been under study for decades or areas that seem like pure fantasy for now.

We've chosen three areas to focus on in this final chapter. The first you've seen a bit of throughout this text: multimodal recommendations. This area is increasingly important as users turn to platforms to do more things. Recall that multimodal recommendations occur when a user is represented by several latent vectors simultaneously.

Next up is graph-based recommenders. We've discussed co-occurrence models, which are the simplest such models for graph-based recommendation systems. They go much deeper! GNNs are becoming an incredibly powerful mechanism for encoding relations between entities and utilizing these representations, making them useful for recommendations.

Finally, we'll turn our attention to large language models and generative AI. During the writing of this book, LLMs have gone from something that a small subset of ML experts understood to something mentioned on HBO

comedy broadcasts. While a rush is occurring to find relevant applications of LLMs to recommendation systems, the industry already has confidence in applying these tools in certain ways. Also exciting, however, is the application of recommendation systems to LLM apps.

Let's see what's coming next!

Multimodal Recommendations

Multimodal recommenders allow for the concession that *users contain multitudes*: a single representation for a user's preferences may not capture the entire story. Someone shopping on a large everything-commerce website, for example, may be all of the following:

- A dog owner who frequently needs items for their dog
- A parent who is always updating the closet for the growing baby
- A hobbyist race-car driver who buys the pieces necessary to drive their car on a track
- A LEGO investor who keeps hundreds of sealed boxes of Star Wars sets hidden away in the closet

The methods you've learned throughout this book should do well at providing recommendations for all of these users. However, you may notice in this list a few areas that are conflicting:

- If your child is very young, why do you buy LEGO sets already? Also, doesn't your dog chew on them?
- If your garage is full of LEGO sets, where do you keep all these car parts?
- Where do you put your dog in that two-seater Mazdaspeed MX-5 Miata?

You can probably think of other cases where some aspects of what you buy just don't match up well with others. This leads to a problem of multimodality: several places in the latent space of your interests coalesce into modes or medoids, but not only one.

Let's return to some of our geometric discussions from before: if you are using nearest neighbors to a user vector, then which of the medoids will take on the most importance?

The way we approach this problem is by multimodality, or providing several vectors associated to a single user. While a naive approach to scaling to consider all the modes for a user would be to simply increase the dimensionality of the model on the item side (to create more areas in which different types of items can be embedded disjointly), this presents serious challenges at scale in terms of training and memory concerns.

One of the first significant works in this area is coauthored by one of this book's authors and introduces an extension to MF to deal with this; see [“Nonlinear Latent Factorization by Embedding Multiple User Interests”](#) by Jason Weston et al. The goal is to build multiple latent factors simultaneously as we did in our other matrix factorization methods, each factor hopefully taking on representation for one of the user's interests.

This is achieved by constructing a tensor that has its third tensor dimension represent each of the latent factors for distinct interests rather than encoding a user item factorization matrix. The factorization is generalized to the tensor case, and the WSABIE loss you saw earlier is used to train.

Building on this work, several years later Pinterest released PinnerSage, as we mentioned in [Chapter 15](#). This modifies some of the assumptions of the Weston et al. paper, by not assuming a known number of representations for each user. Additionally, this approach uses graph-based feature representations, which we'll talk more about in the next section. Finally, the last important modification that this method uses is clustering: it attempts to build the modes via clustering in item space.

The basic PinnerSage approach is to do the following:

1. Fix item embeddings (they call these *pins*).
2. Cluster user interactions (unsupervised and unspecified in cardinality).

3. Build cluster representations as the medoid of the cluster embeddings.
4. Retrieve using medoid-anchored ANN search.

PinnerSage is still considered to be near state of the art for large-scale multimodal recommenders. Some systems take another approach to allow users to more directly modify their “mode” by selecting the theme of what they’re looking for, while others hope to learn it from a sequence of interactions.

Next up, we’ll look at how higher-order relationships between items or users can be explicitly specified.

Graph-Based Recommenders

Graph neural networks (GNNs) are a class of neural networks that use the structural information of data to build deeper representations of your data. They’ve proven especially useful when dealing with relational or networked data, both of which have utility.

One moment of disambiguation before we continue: *graphs* in the sense that we will use them here refer to collections of *nodes* and *edges*. These are purely mathematical concepts, but generally we can think of nodes as the objects of interest and edges as the relationships between them. These mathematical objects are useful for distilling down the core of what is necessary for the kind of representation you wish to build. While the objects may seem very simple, we can add just the right amount of complexity in a variety of ways to capture more nuance.

In the simplest setups, each node on the graph represents an item or user, and each edge represents a relationship such as a user’s interaction with an item. However, user-to-user and item-to-item networks are extremely powerful extensions as well. Our co-occurrence models are simple graph networks; however, we did not learn a representation from these and instead directly took these as our models.

Let's consider a few examples of adding more structure to a graph to encode ideas:

Directionality

This ordering on an edge's vertices can be added to indicate a strict relationship of one node acting on the other; e.g., a user *reads* a book but not the other way around.

Edge decorations

Descriptors such as edge labels can be added to communicate features about the relationships; e.g., two users share account credentials, *and one of the users is identified as a child*.

Multiedges

These can allow for relationships to have higher multiplicity, or allow for the same two entities to have multiple relationships. In a graph of outfits with clothing items as nodes, each edge can be another clothing item that makes the other two go well together.

Hyper-edges

A step further up the level of abstraction may add these edges, which connect multiple nodes simultaneously. For video scenes, you may detect objects of various classes, and your graph may have nodes for those classes, but understanding not only which pairs of object classes appear but which higher-order combinations appear can be identified with hyper-edges.

Let's explore the basics of GNNs and how their representations are a bit different.

Neural Message Passing

In GNNs our object of interest is assigned as the nodes in our graph. Usually, the main objective in GNNs is to build powerful representations of the nodes and edges, or both, via their relationships.

The fundamental difference between GNNs and traditional neural networks is that during the training, we’re explicitly using operators that transfer data between node representations “along the edges.” This is called *message passing*. Let’s start with an example to prime the basic idea.

Let nodes represent users, and their features are persona details such as demographic, onboarding survey question, etc. Let edges be the social network graph: are they friends? And let’s add decoration to the edges, such as the number of DMs exchanged between them on the platform. If we are the social media company that wants to introduce ad shopping to our platform, we may start with those persona features, but we’d ideally like to use something about this network of communication. In theory, people who communicate and share content with each other a lot may have similar tastes. Somewhat tellingly, we introduce a concept called a *message function*, which allows features to be sent from node to node. The message function uses features from each node and the edge between them, written mathematically as follows, for $h_i^{(k)}$ the features at node i and $h_j^{(k)}$ at node j , respectively:

$$m_{ij}^{(k)} = \mathcal{M} \left(h_i^{(k)}, h_j^{(k)}, e_{ij} \right)$$

The features of the edge are e_{ij} and $\mathcal{M}$ is some differentiable function. Note that the superscript $^{(k)}$ refers to the layer as is standard in back-prop notation. Here are two simple examples:

- $m_{ij}^{(k)} = h_i^{(k)}$ means “take the features from a neighbor node”
- $m_{ij}^{(k)} = \frac{h_i^{(k)}}{c_{ij}}$ means “average by the number of edges between i and j ”

Many powerful message-passing schemes that use learning use approaches from other areas of ML—like adding an attention mechanism on node features—but this book doesn’t dive deep into this theory.

The next function we’ll introduce is the *aggregation function*, which takes as input the collection of messages and aggregates them. The most common types of aggregation functions do the following:

- Concatenate all the messages
- Sum all the messages
- Average all the messages
- Take the max of the messages

Finally, we will use the output of the aggregation as part of our update function, which takes node features and aggregated message functions and then applies additional transformations. If you’ve been wondering, “Where does this model learn anything?” the answer is in the update function. The update function usually has a weight matrix associated to it, so as you train this neural network, you are learning the weights in the update function. The simplest update functions multiply a weight matrix by the vectorized output of your aggregation and then apply an activation function per vector.

This chain of message passing, aggregating, and updating is the core of GNNs and encompasses a broad capability. They’ve been useful for ML tasks of every kind, including recommendations. Let’s see some direct applications to recommendation systems.

Applications

Let’s revisit some of the high-level ideas that GNNs may touch in the RecSys space.

Modeling user-item interactions

In other methods we’ve presented, such as matrix factorization, the interactions between users and items are considered, but the complex network among users or items is not exploited. In contrast, GNNs can capture the complex connections in the user-item interaction graph and then use the structure of this graph to make more accurate recommendations.

Thinking back to our message passing, it allowed us to “spread” the information of some nodes (in this case, user and items) to their neighbors. An analogy for this would be that as a user interacts more and more with items with specific features, some of those features are imbued onto the

user. This may sound similar to latent features, because it is! These are ultimately helping the network build a latent representation from the messages that pass features from items to user. This can be even more powerful than other latent embedding methods, because you explicitly define the structural relationships and how they communicate these features.

Feature learning

GNNs can learn more expressive feature representations of nodes (users or items) in a graph by aggregating feature information from their neighbors, leveraging the connections between nodes. These learned features can provide rich information about users' preferences or items' characteristics, which can greatly enhance the performance of recommendation systems.

Previously, we talked about how a user's representations can learn from the items they interact with, but items can also learn from one another. Similar to the way item-item collaborative filtering (CF) allows items to pick up latent features from shared users, GNNs allow us to add potentially many other direct relationships between items.

Cold-start problem

Recall our cold-start problem: providing recommendations for new users or items is difficult because of the lack of historical interactions. By using the features of nodes and the structure of the graph, GNNs can learn the embeddings for new users or items, potentially alleviating the cold-start problem.

In some of our graphical representations of our user graph, the edges need not only exist between users with lots of prior recommendations. It's possible to use other user actions to *bootstrap* some early edges. Structural edges like “share a physical location” or “invited by the same user” or “answers onboarding questions similarly” can be enough to quickly bootstrap several user-user edges, which allow us to warm-start recommendations for them.

Context-aware recommendations

GNNs can incorporate contextual information into the recommendation process. For example, in a session-based recommendation, a GNN can model the sequence of items a user has interacted with in a session as a graph, where each item is a node and the sequential order forms edges. The GNN can then learn the dynamic and complex transitions among items to make context-aware recommendations.

These high-level ideas should point to the opportunity in graph encoding for recommender problems, but let's look at two specific applications next: random walks and metapaths.

Random Walks

Random walks in GNNs enable methods to use the user-item interaction graph to learn effective node (i.e., user or item) embeddings. The embeddings are then used to make recommendations. In the context of graphs, a random walk is an iterative process of starting on a particular node and then stochastically moving to another connected node via a randomized choice.

One popular random-walk-based algorithm for network embedding is DeepWalk, which has been adapted and extended in many ways for various tasks, including recommendation systems.

Here's how a random-walk GNN approach might work in a recommendation context:

1. Random walks generation: start by performing random walks on the interaction graph. Starting from each node, make a series of random steps to other connected nodes. This results in a set of paths, or "walks," that represent the relationships between different nodes.
2. Node embeddings: the sequences of nodes generated by the random walks are treated similar to sentences in a corpus of text, and each node is treated like a word. Word2vec or similar language-modeling techniques are then used to learn embeddings for the nodes (vector

representations), such that nodes appearing in similar contexts (in the same walks) have similar embeddings.

3. Recommendations: once you have learned node embeddings, you can use them to make recommendations. For a given user, you might recommend items that are “close” to that user in the embedding space, according to a distance metric. This can use all the techniques we’ve previously developed for recommendations from latent space representations.

This approach has some nice properties:

- It can capture the high-order connections in the graph. Each random walk can explore a part of the graph that’s not directly connected to the starting node.
- It can help with the sparsity problem in recommender systems because it uses the graph’s structure to learn representations, which requires less interaction data.
- It naturally attempts to handle cold-start issues. For new users or items with few interactions, their embeddings can be learned from connected nodes.

Nevertheless, this approach has some challenges. Random walks can be computationally expensive on large graphs, and it might be difficult to choose appropriate hyperparameters, such as the length of the random walks. Also, this approach may not work as well for dynamic graphs, where interactions change over time, since it doesn’t inherently consider temporal information.

This method implicitly assumes that the nodes are heterogeneous, and so co-embedding them via connections is natural. While it was not an explicit requirement, the type of sequence embeddings DeepWalk builds tends to structurally assume this. Let’s break this rule to accommodate learning between heterogeneous types in our next architecture example, metapaths.

Metapath and Heterogeneity

Metapath was introduced to improve explainable recommendations and integrate the ideas of knowledge graphs with GNNs.

A *metapath* is a path in a heterogeneous network (or graph) that connects different types of nodes via different types of relationships.

Heterogeneous networks contain various types of nodes and edges, representing multiple types of objects and interactions. Beyond simply users and items, the node types can be “carts of items” or “viewing sessions” or “channel used for purchase.”

Metapaths can be used in GNNs for handling heterogeneous information networks (HINs). These networks provide a more comprehensive representation of the real world. When used in a GNN, a metapath provides a scheme for the way information should be aggregated and propagated through the network. It defines the type of paths to be considered when pooling information from a node’s neighborhood.

For example, in a recommender system, you might have a heterogeneous network with users, movies, and genres as node types, and “watches” and “belongs to” as edge types. A metapath could be defined as “User - watches → Movie - belongs to → Genre - belongs to → Movie - watches → User.” This metapath represents a way of connecting two users through the movies they watch and the genres of those movies.

A popular method that utilizes metapaths is the heterogeneous GNN (Hetero-GNN) and its variants. These models leverage the metapath concept to capture the rich semantics in HINs, enhancing the learning of node representations.

Metapath-based models have shown promising results in various applications, as they allow you to explicitly encode much more abstract relationships into the message-passing mechanisms we’ve mentioned.

If higher-order modeling is your thing, buckle up for the last concept we’ll cover in this book. This topic is state of the art and full of high-level ab-

stractions. Language-model-backed agents are at the absolute cutting edge of ML modeling.

LLM Applications

All of the superlatives for LLMs have been used up. For that reason, we'll just say this: LLMs are powerful and have a surprisingly large number of applications.

LLMs are general models that allow users to interact with them via natural language. Fundamentally, these models are generative (they write text) and auto-regressive (what they write is determined by what came before). Because LLMs can speak conversationally, they've been branded as general artificial *agents*. It's natural to then ask, "Can an agent recommend things for me?" Let's start by examining how to use an LLM to make recommendations.

LLM Recommenders

Natural language is a wonderful interface to ask for recommendations. If you want a coworker's recommendation for lunch, maybe you'll show up at their desk and say nothing—hoping they'll remember their latent knowledge of your preferences, identify the time-of-day context, recall the availability of restaurants based on day-of-week, and keep in mind that yesterday you had a pastrami sandwich.

More effectively, you could simply ask, "Any suggestions for lunch?"

Like your astute coworker, models may be more effective at providing recommendations if you simply ask them to. This approach also adds the capability of defining more precisely the kind of recommendation you want. A popular application of LLMs is to ask them for recipes that use a set of ingredients. Thinking through this in the context of the kind of recommenders we've built, building a recommender of this kind has some hurdles. It probably needs some user modeling, but it's very dependent on the items specified. This means that there's a very low signal for each combination of specified items.

An LLM, on the other hand, is quite effective at the autoregressive nature of this task: given a few ingredients, what's most likely to be included next in the context of a recipe. By generating several items like this, a ranking model can augment this to provide a realistic recommender.

LLM Training

Large generative language models of the type that have exploded in popularity are trained in three stages:

1. Pretraining for completion
2. Supervised fine-tuning for dialogue
3. Reinforcement learning from human feedback

Sometimes the latter two steps are combined into what is called *Instruct*. For an exceptionally deep dive into this topic, see the original InstructGPT paper [“Training Language Models to Follow Instructions with Human Feedback”](#) by Long Ouyang et al.

Let's recall that text-completion tasks are equivalent to training the model to predict the correct word in a sequence after seeing k previous ones. This may remind you of GloVe from [Chapter 8](#), or our discussion about sequential recommenders.

Next up is fine-tuning for dialogue; this step is necessary to teach the model that the “next word or phrase” should sometimes be a response instead of an extension of the original statement.

During this stage, the data used for this training is in the form of *demonstration data*, i.e., pairs of statements and responses. Examples include the following:

- A request and then a response to that request
- A statement and then a translation of that statement
- A long text and then a summarization of that text

For recommendations, you can imagine that the first is highly relevant to the task we hope the model to demonstrate.

Finally, we move to the reinforcement learning from human feedback (RLHF) stage; the goal here is to learn a reward function that we can later use to further optimize our LLM. However, the reward model *itself* needs to be trained. Interestingly for recommendation systems enthusiasts like yourself, AI engineers do this via a ranking dataset.

A large number of tuples—similar to the demonstration data we’ve seen—provide statements and responses, although instead of only one response, there are multiple responses. They are ranked (via a human labeler), and then for each pair of superior-inferior responses (x, sup, inf) , we evaluate the loss:

- $r_{sup} = \Theta(x, sup)$ is the reward model’s score for the superior response.
- $r_{inf} = \Theta(x, inf)$ is the reward model’s score for the inferior response.

The final loss is computed: $-\log(\sigma(sup - inf))$.

This reward function is then used to fine-tune the model.

OpenAI summarizes this approach via the diagram in [Figure 18-1](#).

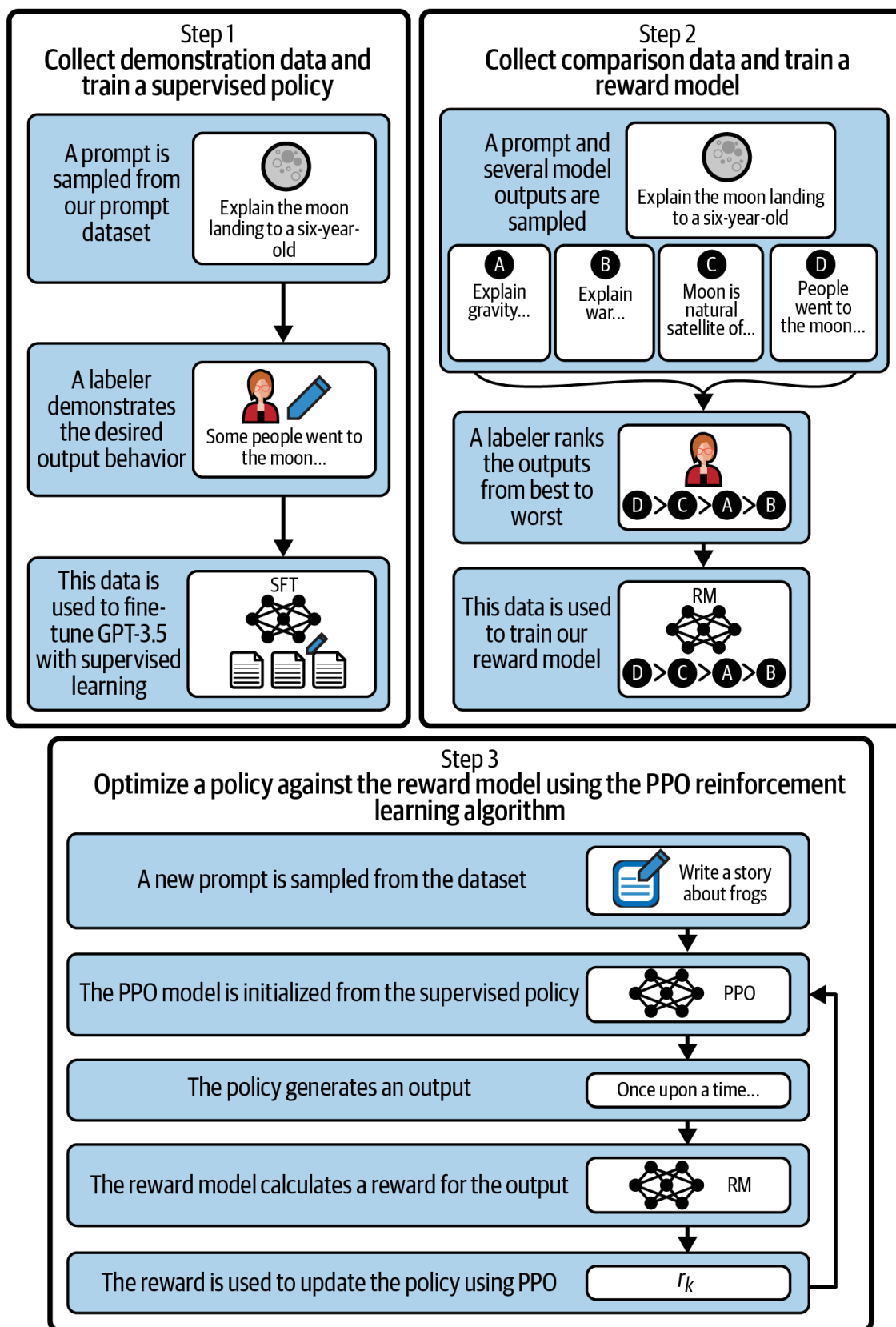


Figure 18-1. Instruct methodology for model fine-tuning

From this brief overview, you can see that these LLMs are trained to respond to requests—something well suited for a recommender. Let’s see how to augment this training.

Instruct Tuning for Recommendations

In the previous discussion of instruct pairs, we saw that ultimately the aim of the training was to learn a rank comparison between two responses. This kind of training should feel quite familiar. In [“TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation”](#) by Kegin Bao et al., the authors use a similar setup to teach user preferences to the model.

As the paper mentions, historical interaction items are collected into two groups based on their ratings: user likes and user dislikes. They collect this information into natural language prompts to format a final “Rec Input”:

1. User preference: $[item_1, \dots, item_n]$
2. User preference: $[item_1, \dots, item_n]$
3. Will the user enjoy the User preference, $[item_{n+1}]$?

These follow the same training pattern as InstructGPT noted previously. The authors achieve dramatically improved performance on recommender problems as compared to an untrained LLM for recommendations; however, those should be considered baselines as it’s not their target task.

LLM Rankers

So far in this chapter, we’ve thought of the LLM as a recommender in totality, but instead, the LLM can be used as simply the ranker. The most trivial approach to this is to simply prompt the LLM with the relevant features of a user and a list of items and ask it to suggest the best options.

While naive, variants on this approach have seen somewhat surprising results in very generic settings: “The user wants to watch a scary movie tonight and isn’t sure which will be the best if he doesn’t like gore: movie-1, movie-2, etc.” But we can do better.

Ultimately, as with LTR approaches, we can think of pointwise, pairwise, and listwise. If we wish to use an LLM for a pointwise ranking, we should constrain our prompting and responses to a setting in which these models may be useful. Take, for example, a recommender for scientific papers; a user may wish to write what they're working on and have the LLM helpfully suggest papers of relevance. While a traditional search problem, this is a setting in which our modern tools can bring a lot of utility: LLMs are effective at summarizing and semantic matching, which means that semantically similar results may be found from a large corpus, and then the agent can synthesize the output of those results into a cogent response. The biggest challenge here is hallucination, or suggesting papers that may not exist.

You can think of pairwise and listwise similarly: distilling the reference data into a shape that the unique capabilities of these LLMs can use to make significant assists.

While we're near the topic of search and retrieval, it's important to mention one of the ways in which recommendation can help LLM applications: retrieval augmentation.

Recommendations for AI

We've seen how LLMs can be used to generate recommendations, but how do recommenders improve LLM applications? LLM agents are extremely general in their capabilities but lack specificity on many tasks. If you ask an agent, "Which of the books I read this year were written by nonwestern authors?" the agent has no chance of success. Fundamentally, this is because the general pretrained models have no idea what books you've read this year.

To solve for this, you'll want to leverage *retrieval augmentation*, i.e., providing relevant information to the model from an existing data store. The data store may be an SQL database, a lookup table, or a vector database, but ultimately the important component here is that somehow from your request, you're able to find relevant information and then provide it to an agent.

One assumption we’ve made here is that your request is interpretable by your retrieval system. In the preceding example, you’d like the system to automatically understand the “which of the books I read this year” phrase as an information-retrieval task equivalent to something like this:

```
SELECT * FROM read_books
WHERE CAST(finished_date, YEAR) = CAST(today(), YEAR)
```

Here we’ve just made up an SQL database, but you can imagine schema to satisfy this request. Converting from the request to this SQL is now yet another task you need to model—maybe it’s the job of another agent request.

In other contexts, you want a full-scale recommender to help with the retrieval: if you want users to ask an agent for a movie tonight, but also to continue to use your deep understanding of each user’s tastes, you could first filter the potential movies by the user’s preference and then send only movies your recommender model thinks are great for them. The agent can then service the text request from a subset of movies that are already determined to be great.

The intersection of LLMs and recommendation systems is going to dominate much of the conversation in recommendation systems for a while. There’s a lot of low-hanging fruit in bringing the knowledge of recommender systems to this new industry. As Eugene Yan recently said:

I think the key challenge, and solution, is getting them [LLMs] the right information at the right time. Having a well-organized document store can help. And by using a hybrid of keyword and semantic search, we can accurately retrieve the context that LLMs need.

Summary

The future of recommendation systems is bright, but the technology will continue to get more complicated. One of the major changes over the last five years has been an incredible shift to GPU-based training and the ar-

chitectures that can use these GPUs. This is the primary motivation for why this book favors JAX over TensorFlow or Torch.

The methods in this chapter embrace bigger models, more interconnections, and potentially inference on a scale that's hard to house in most organizations. Ultimately, recommendation problems will always be solved via the following:

- Careful problem framing
- Deeply relevant representations of users and items
- Thoughtful loss functions that encode the nuances of the task
- Great data collection